

ECE 220 Computer Systems & Programming

Lecture 3 — The Stack

September 1, 2026

Prof. Sayan Mitra `mitras@illinois.edu`

Course website: `courses.grainger.illinois.edu/ece220/fa2026`

 **ILLINOIS**

Electrical & Computer Engineering

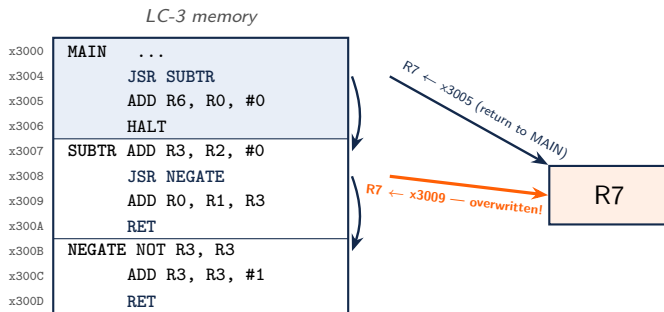
GRAINGER COLLEGE OF ENGINEERING

Announcements

- **Mock quiz at the CBTF: Sep 7–10** — extra credit, full 25 points just for *participating*
- Use it to try out the testing screens and options (PrairieLearn, the LC-3 web simulator) — so there are **no surprises** during a real quiz
- Register now at `cbtf.illinois.edu/students`
(each quiz can be scheduled up to 10 days in advance)
- **Midterm 1: Thursday, October 1, 7:00–8:20 pm** — in person, paper, covers Lectures 1–6
conflict-exam sign-up deadline: Sep 27 (form on the exams page)

Problem with Saving Register Values in Nested Subroutine Calls

One R7, many nested calls — where do all the return addresses go?



Not unique to R7 — same problem with saving *any* register values, if a subroutine calls itself.

Approach: Storage that Grows with Each Call, in Reverse Order

1. Which data structure returns things in exactly *reverse* order?
2. How do we build one from plain memory and a single register?

Approach: Storage that Grows with Each Call, in Reverse Order

1. Which data structure returns things in exactly *reverse* order?
2. How do we build one from plain memory and a single register?



Level 1: rainy city (Yusuf's dream)



Level 2: the hotel (Arthur's dream)



Level 3: snow fortress (Eames's dream)

Inception (C. Nolan, 2010), © Warner Bros.

Approach: Storage that Grows with Each Call, in Reverse Order

1. Which data structure returns things in exactly *reverse* order?
2. How do we build one from plain memory and a single register?

Today: the stack — one of the most used data structures in computing.
Yes, *that* stack: stackoverflow.com is named after the error you get when nested calls run it out of room.



Level 1: rainy city (Yusuf's dream)



Level 2: the hotel (Arthur's dream)



Level 3: snow fortress (Eames's dream)

Inception (C. Nolan, 2010), © Warner Bros.

Warm-up: Would This Program Work?

```
.ORIG x3000
    AND  R1, R1, #0
    AND  R2, R2, #0
    ADD  R1, R1, #5    ; R1 = 5
    ADD  R2, R2, #2    ; R2 = 2
    JSR  SUBTR        ; R0 ← R1 - R2
    ADD  R6, R0, #0    ; copy result
    HALT
```

```
; SUBTR: R0 ← R1 - R2
SUBTR  ADD  R3, R2, #0
        JSR  NEGATE    ; R3 = -R2
        ADD  R0, R1, R3 ; R0 = R1 - R2
        RET
; NEGATE: R3 ← -R3
NEGATE NOT  R3, R3
        ADD  R3, R3, #1
        RET
```

Trace it: `JSR NEGATE` overwrites `SUBTR`'s return address in `R7`. `SUBTR`'s `RET` jumps back to just after `JSR NEGATE` — forever. Infinite loop!

Fix: `SUBTR` must save `R7` before its own `JSR` — but saving to labeled slots can't handle *arbitrary* nesting. We need something better.

Work with a neighbor: does the program reach `HALT`?
What is in `R0`?

Stack — An Abstract Data Type

A **LIFO (last-in, first-out)** storage structure:

- The **first** thing you put in is the last thing you take out
- The **last** thing you put in is the first thing you take out

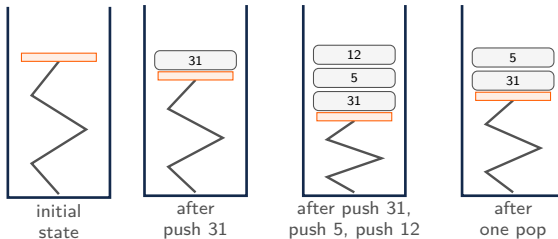
The **means of access** is what defines a stack — not the implementation.

- **Push**: add an item to the top
- **Pop**: remove the item at the top
- **IsFull**: a push when full is an overflow
- **IsEmpty**: a pop when empty is an underflow

What else in computing behaves like a stack?

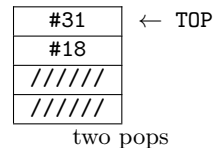
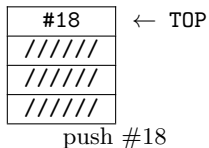
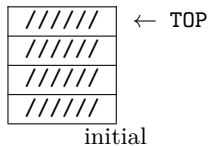
A Physical Stack: the Coin Holder

First coin in is the **last** coin out.



Implementation in Hardware: Data Moves

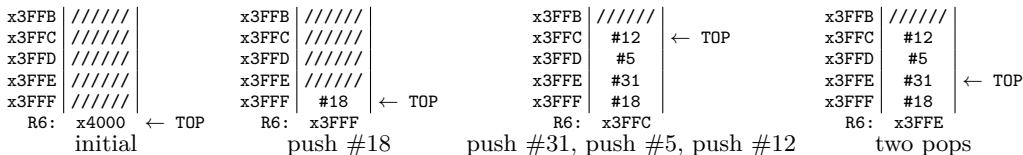
Data items move; the top of the stack is a fixed location.



Fine for hardware (shift registers) — wasteful in memory: every push would copy every element.

Implementation in Memory: the Pointer Moves

Data items stay put; only our idea of where the top is moves.



- By convention, **R6** holds the **Top-of-Stack (TOS)** pointer
- The stack **grows toward lower addresses**

After the pops, #12 and #5 are still in memory — just no longer reachable by stack rules.

Basic Push and Pop

With R6 pointing at the top element:

Push (R0 holds the value):

ADD R6, R6, #-1 ; make room (grow downward)

STR R0, R6, #0 ; store at the new top

Pop (value comes back in R0):

LDR R0, R6, #0 ; read the top

ADD R6, R6, #1 ; shrink the stack

Two instructions each — but what happens if we push when full, or pop when empty?

A Safe PUSH: Setup

x3FF0		← STACK_END
⋮		
x3FFE		
x3FFF		
x4000		← STACK_START

16 usable slots: x3FF0–x3FFF.

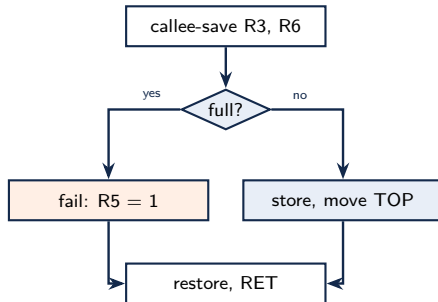
`STACK_TOP` lives in *memory* — the subroutine loads it into R6, uses it, and stores it back.

```
STACK_START .FILL x4000
STACK_END   .FILL x3FF0
STACK_TOP    .FILL x4000    ; empty: TOP = START
P_SAVER3     .BLKW #1
P_SAVER6     .BLKW #1
```

The contract:

- **IN:** R0 = value to push
- **OUT:** R5 = 0 (success) or 1 (overflow)
- Everything else: callee-saved
- Full when TOP = STACK_END

In-Class Problem: PUSH with Overflow Checking



```

PUSH    ST    R3, P_SAVER3
        ST    R6, P_SAVER6
        AND   R5, R5, #0    ; assume success
        LD    R3, STACK_END
        LD    R6, STACK_TOP
        NOT   R3, R3        ; R3 = -END
        ADD   R3, R3, #1
        ADD   R3, R6, R3    ; TOP - END
        BRz   P_OVER       ; full -> fail
        ADD   R6, R6, #-1   ; grow downward
        STR   R0, R6, #0    ; store the value
        ST    R6, STACK_TOP
        BR    P_DONE
P_OVER  ADD   R5, R5, #1    ; R5 = 1: overflow
P_DONE  LD    R3, P_SAVER3
        LD    R6, P_SAVER6
        RET
  
```

How do we compare TOP with STACK_END? **Negate and add — LC-3 has no compare instruction.**

A Safe POP: Setup

x3FF0		← STACK_END
⋮		
x3FFE		
x3FFF		
x4000		← STACK_START

Underflow: popping from an *empty* stack
— there is no top element to read.

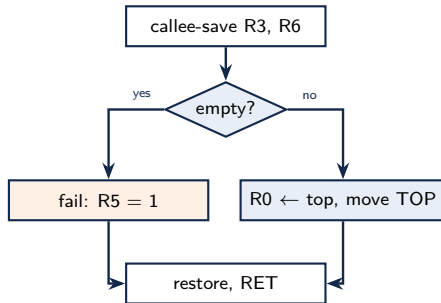
Same discipline as PUSH: load `STACK_TOP`
into R6, use it, store it back.

```
STACK_START .FILL x4000
STACK_END   .FILL x3FF0
STACK_TOP   .FILL x4000    ; empty: TOP = START
Q_SAVER3    .BLKW #1
Q_SAVER6    .BLKW #1
```

The contract:

- **IN:** nothing
- **OUT:** R0 = the popped value; R5 = 0 (success) or 1 (underflow)
- Everything else: callee-saved
- Empty when TOP = STACK_START

In-Class Problem: POP with Underflow Checking



```

POP      ST    R3, Q_SAVER3
         ST    R6, Q_SAVER6
         AND   R5, R5, #0    ; assume success
         LD    R3, STACK_START
         LD    R6, STACK_TOP
         NOT   R3, R3        ; R3 = -START
         ADD   R3, R3, #1
         ADD   R3, R6, R3    ; TOP - START
         BRz   Q_UNDER      ; empty -> fail
         LDR   R0, R6, #0    ; R0 ← top value
         ADD   R6, R6, #1    ; shrink the stack
         ST    R6, STACK_TOP
         BR    Q_DONE
Q_UNDER  ADD   R5, R5, #1    ; R5 = 1: underflow
Q_DONE  LD     R3, Q_SAVER3
         LD     R6, Q_SAVER6
         RET
  
```

How do we detect *empty*? $TOP - STACK_START = 0$ — the same negate-and-add trick as PUSH.

Summary & What's Next

Today

- **The problem:** nested calls need somewhere safe to put return addresses and saved registers
- **The stack ADT:** LIFO; push, pop, overflow, underflow — defined by the access rule, not the implementation
- **In memory:** data stays put, **R6** (the TOS pointer) moves; the stack grows toward lower addresses
- **Safe PUSH & POP:** check for overflow/underflow, report success/failure in R5, callee-save everything you touch

Next lecture

- Programming *with* the stack: evaluating arithmetic expressions

Code from today: `nested_bug.asm`, `push.asm`, `pop.asm` in the course code repository.